



# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

*Facultad de Ciencias*

---

## Sistemas Operativos

ZFS & BtrFS

---

## Sistemas de Archivos Tarea 6

*Presenta:*

**Rojó Peña Manuel Ianluck - 118005762**

*Profesor:*

**Javier León Cotonieto**

*Ayudante laboratorio:*

**Fernanda Garduño Ballesteros**

Grupo: 7004, 2026-2

*Fecha de entrega:*

8 de mayo, 2026

---

## 1. Introducción

---

Los sistemas de archivos son uno de los componentes más críticos de cualquier sistema operativo, determinan cómo se almacenan, organizan, protegen y recuperan los datos en los medios físicos. A lo largo de las últimas décadas, la demanda de mayor capacidad, integridad y flexibilidad impulsó el surgimiento de sistemas de archivos de nueva generación que van mucho más allá de los tradicionales FAT, NTFS o ext4.

- **ZFS (Zettabyte File System)**: concebido por Sun Microsystems en 2001 y lanzado en 2005, revolucionó el concepto de almacenamiento empresarial al fusionar el sistema de archivos con el gestor de volúmenes, ofreciendo integridad de datos de extremo a extremo y capacidades casi ilimitadas.
- **Btrfs (B-Tree File System)**: propuesto en 2007 y adoptado por el kernel Linux en 2009, fue diseñado como el sucesor natural de ext4, trayendo instantáneas, sub-volúmenes y compresión transparente de forma nativa al ecosistema Linux.

---

## 2. ZFS (Zettabyte File System)

---

ZFS fue diseñado e implementado por un equipo de ingenieros de **Sun Microsystems** liderado por **Jeff Bonwick** y **Matthew Ahrens**, el proyecto inició formalmente en **2001** con el objetivo de crear un sistema de archivos radicalmente diferente para el sistema operativo **Solaris**. Las siglas originalmente significaban *Zettabyte File System*, aunque posteriormente el proyecto adoptó el nombre como acrónimo recursivo.

El anuncio oficial de ZFS se produjo en **septiembre de 2004**, el código fuente fue integrado en la rama principal de desarrollo de Solaris el 31 de octubre de 2005 y puesto en producción el **16 de noviembre de 2005** como parte de la compilación 27 de OpenSolaris.

En 2010, tras la adquisición de Sun por parte de Oracle, el proyecto de código abierto continuó a través de la comunidad bajo el nombre **OpenZFS** (fundado en 2013), que hoy mantiene la implementación para Linux (**ZFS on Linux**), FreeBSD, macOS y otros sistemas.

**Licencia:** ZFS fue originalmente liberado bajo la **CDDL** (*Common Development and Distribution License*), que es código abierto pero incompatible con la GPL. Por esta razón, ZFS no puede incluirse directamente en el kernel Linux (que usa GPL), sino que se distribuye como módulo externo mediante el proyecto **OpenZFS / ZFS on Linux**.

### 2.1. Propósito y problema que resolvía

Los sistemas de archivos de la época (UFS, ext2, ext3) requerían una capa separada de gestión de volúmenes (LVM, mdadm) para manejar múltiples discos, lo que generaba:

- Complejidad administrativa por capas independientes.
- Falta de visibilidad del sistema de archivos sobre la disposición física de los datos.
- Incapacidad para detectar y corregir la *corrupción silenciosa (bit rot)*, es decir, la degradación de datos en disco sin que ningún sistema de nivel superior lo advirtiera.
- Necesidad de ejecutar `fsck` tras cada apagado abrupto, sin garantía de éxito.

ZFS fue diseñado para resolver todos estos problemas de raíz, fusionando el gestor de volúmenes y el sistema de archivos en una sola entidad coherente con visibilidad completa sobre el hardware.

## 2.2. Innovación principal

La innovación de ZFS es su modelo de **integridad de extremo a extremo basado en árboles de Merkle**. Cada bloque de datos posee un *checksum* (suma de comprobación) de 256 bits almacenado no junto al bloque, sino en el *puntero del bloque padre*, formando un árbol jerárquico hasta la raíz. Cuando ZFS lee un dato, recalcula el *checksum* al vuelo; si no coincide, y el pool tiene redundancia, repara el bloque corrupto automáticamente sin intervención del administrador.

## 2.3. Código abierto y propiedad

- **Creador original:** Sun Microsystems (hoy Oracle).
- **Proyecto activo:** OpenZFS (comunidad independiente desde 2013).
- **Licencia:** CDDL (código abierto, incompatible con GPL).
- **Estado:** Software libre de código abierto con gobernanza comunitaria.

## 2.4. Sistemas operativos donde es más usado

ZFS es el sistema de archivos nativo de **FreeBSD** (donde está integrado directamente en el kernel) y de distribuciones derivadas como **TrueNAS** (anteriormente FreeNAS) e **illumos** (sucesor abierto de OpenSolaris). En Linux se usa a través del módulo `zfs-dkms` o `zfs-linux`, popularizado por distribuciones como **Ubuntu** (que lo soporta oficialmente desde 19.10) y **Proxmox VE**.

## 2.5. Uso típico

- Servidores empresariales de almacenamiento (*storage servers*).
- Centros de datos y entornos *cloud* privados.
- Arreglos NAS de alta capacidad y alta disponibilidad (TrueNAS).

- Entornos de virtualización (Proxmox, bhyve) donde la integridad y los snapshots son críticos.
- Sistemas de respaldo y archivo a largo plazo.

## 2.6. Arquitectura e implementación

ZFS no es únicamente un sistema de archivos; es al mismo tiempo un **gestor de volúmenes** y un **controlador RAID** fusionados. Al prescindir de capas intermedias, ZFS tiene visibilidad y control absolutos sobre los discos físicos.

### 2.6.1. Storage Pools (zpool)

En lugar de crear particiones rígidas, ZFS agrupa los dispositivos físicos en un *pool* de almacenamiento. Los sistemas de archivos (*datasets*) se crean dentro del pool y comparten el espacio libre dinámicamente. Los pools pueden configurarse en distintos modos:

- **Simple (stripe)**: uno o más dispositivos sin redundancia.
- **Mirror**: dos o más dispositivos en espejo.
- **RAID-Z1 / Z2 / Z3**: equivalentes a RAID-5, RAID-6 y RAID-7, pero sin el defecto del “agujero de escritura” (*write hole*) gracias al modelo *copy-on-write*.

Adicionalmente, es posible agregar dispositivos de caché de lectura (*L2ARC*) y de registro de escritura síncrona (*ZIL/SLOG*), habitualmente SSD, para mejorar el rendimiento sin sacrificar la integridad.

### 2.6.2. Datasets y zvols

Dentro de un *zpool* se pueden crear dos tipos de *datasets*:

- **Filesystem**: equivalente a un sistema de archivos tradicional con punto de montaje configurable, cuotas y reservas de espacio.
- **Zvol**: dispositivo de bloques virtual accesible desde `/dev/zvol`, útil como disco virtual para máquinas virtuales o como volumen de intercambio (*swap*).

### 2.6.3. Copy-on-Write (CoW) transaccional

ZFS nunca sobrescribe datos existentes. Cuando se modifica un bloque, se escribe en un *nuevo* espacio del disco; luego los punteros de metadatos son actualizados. Este mecanismo garantiza que **el sistema de archivos nunca queda en un estado inconsistente**, eliminando la necesidad de herramientas como `fsck`.

#### 2.6.4. Instantáneas (*Snapshots*) y clones

Como ZFS no sobrescribe datos, crear una instantánea simplemente consiste en *no liberar* los bloques que pertenecían a la versión anterior. Las instantáneas son de solo lectura, casi instantáneas en tiempo y de costo casi nulo en espacio inicial. A partir de una instantánea se puede crear un *clone*, que es una copia escribible que comparte bloques con el origen hasta que divergen.

#### 2.6.5. Autorreparación (*Self-healing*)

En pools con redundancia (mirror o RAID-Z), cada lectura verifica el *checksum* del bloque. Si se detecta corrupción, ZFS obtiene los datos correctos desde la copia redundante y repara el bloque dañado automáticamente. El comando `zpool scrub` permite un escaneo proactivo de todo el pool.

#### 2.6.6. Caché adaptativo (ARC)

El *Adaptive Replacement Cache* (ARC) es el algoritmo de caché en RAM de ZFS. Combina los datos usados más recientemente (LRU) con los usados con mayor frecuencia (LFU), adaptándose en tiempo real a la carga de trabajo, lo que lo hace notablemente más eficiente que el caché de página estándar del kernel.

#### 2.6.7. Compresión y deduplicación

ZFS soporta compresión transparente por dataset (algoritmos LZ4, GZIP, ZSTD). También ofrece deduplicación en línea (*inline deduplication*), aunque esta característica tiene un alto costo en RAM y se recomienda solo en entornos con datos muy redundantes y suficiente memoria.

### 2.7. Capacidades y límites teóricos

ZFS es un sistema de archivos de **128 bits**, lo que le confiere límites prácticamente imposibles de alcanzar en la práctica:

| Parámetro                                              | Límite                                            |
|--------------------------------------------------------|---------------------------------------------------|
| Tamaño máximo de archivo                               | 16 Exabytes                                       |
| Tamaño máximo de sistema de archivos                   | 16 Exabytes                                       |
| Tamaño máximo de <i>zpool</i>                          | $\approx 2^{78}$ bytes ( $\approx 256$ Zebibytes) |
| Número máximo de snapshots                             | $2^{48}$                                          |
| Número máximo de archivos por sistema de archivos      | $2^{48}$                                          |
| Número máximo de sistemas de archivos por <i>zpool</i> | $2^{64}$                                          |

Jeff Bonwick, arquitecto jefe de ZFS, describió la magnitud de estas cifras señalando que llenar un sistema de archivos de 128 bits excedería los límites físicos cuánticos de almacenamiento del planeta.

## 2.8. Ventajas

- Integridad de datos garantizada de extremo a extremo (checksums en árbol).
- Autorreparación transparente sin intervención del administrador.
- Administración unificada de volúmenes y sistema de archivos.
- Instantáneas y clones casi instantáneos y eficientes en espacio.
- Compresión transparente con mejora efectiva del rendimiento de I/O.
- Alta escalabilidad: capacidad prácticamente ilimitada.
- RAID-Z sin el defecto del *write hole*.

## 2.9. Inconvenientes

- Alto consumo de RAM (se recomienda mínimo 8 GB; la deduplicación puede requerir decenas de GB).
- Licencia CDDL incompatible con GPL: no se puede incluir en el kernel Linux oficialmente.
- La naturaleza CoW puede provocar fragmentación elevada a largo plazo.
- No dispone de herramienta de desfragmentación; la solución es migrar los datos a otro *pool*.
- Curva de aprendizaje pronunciada para administradores no familiarizados.

---

## 3. Btrfs (B-Tree File System)

---

### 3.1. Orígenes e historia

La estructura de datos central de Btrfs los *copy-on-write B-trees* fue propuesta originalmente por el investigador de IBM **Ohad Rodeh** en una presentación en la conferencia **USENIX de 2007**. Poco después, **Chris Mason**, entonces en Oracle, tomó esa idea como base e inició el desarrollo activo del sistema de archivos.

Oracle anunció formalmente el proyecto en **2007** con el objetivo declarado de reemplazar a ext4 como sistema de archivos estándar de Linux. El soporte experimental fue incluido en el **kernel Linux 2.6.29** en **enero de 2009**, y la versión 0.19 fue publicada como software libre bajo **licencia GPL** en junio del mismo año.

A diferencia de ZFS, Btrfs fue diseñado *desde el principio* para integrarse en el kernel Linux, aprovechando sus abstracciones de dispositivos de bloque y su subsistema de memoria virtual.

**Nota de estabilidad:** Btrfs se considera estable en sus funciones principales desde aproximadamente 2014. Sin embargo, algunas características avanzadas (como RAID 5/6 y la deduplicación en línea) aún se consideran experimentales o con advertencias en ciertas distribuciones. El formato de disco no ha cambiado de forma incompatible desde su estabilización. Synology lo adoptó para sus dispositivos NAS con DSM 6.

### 3.2. Propósito y problema que resolvía

Ext4, el sistema de archivos dominante en Linux, fue diseñado como una evolución incremental de ext3 y arrastraba limitaciones estructurales importantes:

- Número máximo de inodos fijado en el momento del formateo (sin posibilidad de expansión dinámica).
- Sin soporte nativo para instantáneas ni subvolúmenes.
- Sin checksums de datos (solo de metadatos en algunas implementaciones).
- Límite de tamaño de archivo y volumen de 16 TB en configuraciones típicas.
- Sin compresión transparente integrada.

Btrfs fue diseñado para superar todas estas limitaciones, centrándose especialmente en la **tolerancia a fallos**, la **facilidad de administración** y la **eficiencia del espacio**.

### 3.3. Innovación principal

La innovación central de Btrfs es su **modelo uniforme de árboles B con copy-on-write**. Absolutamente todo dentro del sistema de archivos (metadatos, datos, directorios, listas de extensiones) se modela como nodos u hojas dentro de un bosque de árboles B. Este diseño proporciona:

- Búsquedas logarítmicas  $O(\log n)$  incluso con millones de archivos en un directorio.
- Capacidad de crear subvolúmenes y snapshots de forma casi instantánea.
- Base estructural para el modelo CoW sin necesidad de capas adicionales.

Theodore Ts'o, desarrollador de ext3 y ext4, reconoció que Btrfs incorpora ideas de diseño de Reiser4 y funcionalidades conceptualmente cercanas a las de ZFS, pero implementadas de forma nativa en Linux.

### 3.4. Código abierto y propiedad

- **Creador original:** Oracle Corporation (basado en trabajo de IBM Research).
- **Mantenimiento actual:** Comunidad de kernel Linux y múltiples empresas (Meta, SUSE, Synology).
- **Licencia:** **GPL v2** (totalmente compatible con el kernel Linux).
- **Estado:** Software libre de código abierto nativo del kernel Linux.

### 3.5. Sistemas operativos donde es más usado

Btrfs es exclusivo de **Linux**. Es el sistema de archivos por defecto en:

- **Fedora** (predeterminado desde Fedora 33).
- **openSUSE / SUSE Linux Enterprise** (predeterminado con snapper para rollbacks del sistema).
- **Synology DSM** (en modelos NAS compatibles, como alternativa a ext4).
- Adoptado también en dispositivos embebidos con Linux y algunas distribuciones orientadas a escritorio.

### 3.6. Uso típico

- Estaciones de trabajo Linux con rollbacks de sistema operativo.
- Servidores ligeros y medianos con necesidad de snapshots.
- Dispositivos NAS comerciales (Synology) para protección de datos empresariales.
- Entornos de desarrollo donde se requiere versionado rápido de entornos.
- Sistemas con almacenamiento SSD donde el soporte nativo de TRIM y el modo optimizado resultan ventajosos.

### 3.7. Arquitectura e implementación

#### 3.7.1. Todo es un árbol B

En Btrfs, los inodos, los metadatos de directorios y las extensiones de archivos (*extents*) son nodos u hojas dentro de múltiples árboles B independientes. Cada árbol tiene su propia raíz, y los árboles se interconectan mediante referencias entre árboles. Esta arquitectura permite búsquedas eficientes y facilita la creación de instantáneas duplicando únicamente la raíz del árbol correspondiente.

#### 3.7.2. Copy-on-Write (CoW)

Al igual que ZFS, Btrfs nunca sobrescribe datos en uso. Cuando se modifica un archivo, Btrfs escribe el nuevo contenido en bloques nuevos y actualiza los punteros del árbol B. Los bloques antiguos se liberan únicamente cuando ya no son referenciados por ninguna instantánea. Este mecanismo es la base de las instantáneas eficientes y de la consistencia transaccional.

#### 3.7.3. Subvolúmenes

Los subvolúmenes son el rasgo más distintivo de Btrfs para el usuario de escritorio y servidor. Un subvolumen actúa como un directorio normal pero puede ser:

- Montado de forma independiente con su propio punto de montaje.
- Sujeto a cuotas independientes.
- Objeto de instantáneas individuales sin afectar al resto del sistema de archivos.

Una instalación típica de openSUSE o Fedora crea subvolúmenes separados para `/`, `/home`, `/var/log`, etc., todos compartiendo el mismo espacio de disco físico.

### 3.7.4. Instantáneas (*Snapshots*)

Crear una instantánea en Btrfs consiste en duplicar la raíz del árbol B del subvolumen. No se copian datos; solo se duplican los punteros. El espacio adicional se consume únicamente cuando los datos del subvolumen original divergen de la instantánea. Las instantáneas pueden ser:

- **De solo lectura:** para respaldo o referencia.
- **Escribibles (*writable snapshots*):** lo que permite tener instantáneas de instantáneas, útiles para entornos de prueba.

Synology implementa esta característica permitiendo hasta **256 puntos de recuperación** por hora, día o semana, con recuperación de autoservicio desde el Explorador de archivos de Windows o File Station.

### 3.7.5. Integridad de datos y metadatos duplicados

Btrfs genera *checksums* para todos los datos y metadatos. Por defecto, almacena **dos copias de los metadatos** en el volumen para mayor resiliencia. Durante cada lectura, se verifican los checksums; si se detecta una discrepancia (*silent data corruption*), el sistema puede recuperar los datos correctos desde las copias redundantes de metadatos o desde la configuración RAID configurada (RAID 1, RAID 5, RAID 6, RAID 10).

### 3.7.6. Soporte multi-dispositivo y RAID nativo

Btrfs gestiona múltiples dispositivos de forma nativa, sin necesidad de mdadm ni LVM, soporta:

- RAID 0 (*striping*).
- RAID 1 (*mirroring*).
- RAID 10.
- RAID 5 y RAID 6 (considerados experimentales; se desaconseja en producción crítica).

### 3.7.7. Compresión transparente

Btrfs comprime los bloques de datos al vuelo antes de escribirlos en disco. Los algoritmos disponibles son:

- **zlib:** mayor ratio de compresión, mayor uso de CPU.
- **lzo:** compresión rápida, menor ratio.
- **zstd:** equilibrio óptimo entre velocidad y ratio de compresión (recomendado actualmente).

### 3.7.8. Desfragmentación en línea y modo SSD

A diferencia de ZFS, Btrfs incorpora una herramienta de **desfragmentación sin necesidad de desmontar** el sistema de archivos (`btrfs filesystem defragment`). También dispone de un modo optimizado para SSD (opción de montaje `ssd`) que adapta la estrategia de asignación de bloques para reducir la amplificación de escritura.

### 3.7.9. Asignación dinámica de inodos

A diferencia de ext4, Btrfs **no fija un número máximo de inodos** en el momento del formateo. Los inodos se asignan dinámicamente según la demanda, lo que elimina el error “no space left on device” causado por agotamiento de inodos (un problema conocido de ext4 en sistemas con millones de archivos pequeños).

## 3.8. Capacidades y límites

| Parámetro                                      | Límite             |
|------------------------------------------------|--------------------|
| Tamaño máximo de archivo                       | 16 Exabytes        |
| Tamaño máximo de volumen                       | 16 Exabytes        |
| Número de instantáneas por subvolumen          | Hasta 65.536       |
| Número de subvolúmenes por sistema de archivos | $2^{64}$ (teórico) |
| Tamaño máximo de nombre de archivo             | 255 bytes          |

## 3.9. Ventajas

- Licencia GPL: totalmente integrado en el kernel Linux sin módulos externos.
- Subvolúmenes y snapshots nativos, fundamentales para rollbacks del sistema operativo.
- Asignación dinámica de inodos: sin límite prefijado de número de archivos.
- Desfragmentación en línea sin desmontar.
- Modo optimizado para SSD.
- Compresión transparente (incluyendo `zstd`) con configuración por directorio.
- Adopción activa en distribuciones Linux modernas (Fedora, openSUSE, Synology DSM).

## 3.10. Inconvenientes

- RAID 5 y RAID 6 con problemas conocidos (*write hole*); no recomendados para producción crítica.
- Menor madurez en entornos empresariales de muy alta carga comparado con ZFS o ext4.

- El rendimiento puede degradarse con alta fragmentación en HDDs mecánicos.
- La deduplicación en línea no está disponible nativamente; requiere herramientas externas (`duperemove`, `bees`).
- Algunas herramientas de monitoreo y respaldo del ecosistema Linux todavía no aprovechan completamente las capacidades de Btrfs.

---

## 4. Comparativa entre ZFS y Btrfs

---

## 4.1. Tabla de comparativa general

| Característica                     | ZFS                                                          | Btrfs                                      |
|------------------------------------|--------------------------------------------------------------|--------------------------------------------|
| <b>Año de inicio</b>               | 2001 (lanzado en 2005)                                       | 2007 (kernel en 2009)                      |
| <b>Creador</b>                     | Sun Microsystems                                             | Oracle (IBM Research)                      |
| <b>Mantenimiento actual</b>        | OpenZFS (comunidad)                                          | Kernel Linux                               |
| <b>Licencia</b>                    | CDDL (código abierto)                                        | GPL v2                                     |
| <b>Integración en kernel Linux</b> | Módulo externo ( <code>dkms</code> )                         | Nativa                                     |
| <b>SO donde es más usado</b>       | FreeBSD, TrueNAS, Ubuntu                                     | Linux (Fedora, openSUSE)                   |
| <b>Uso típico</b>                  | Servidores empresariales, NAS de alta gama, centros de datos | Escritorios Linux, servidores medios, etc. |
| <b>Tamaño máx. de archivo</b>      | 16 Exabytes                                                  | 16 Exabytes                                |
| <b>Tamaño máx. de volumen</b>      | $\approx$ 256 Zebibytes                                      | 16 Exabytes                                |
| <b>Gestión de volúmenes</b>        | Integrada (zpool)                                            | Integrada (multi-dispositivo)              |
| <b>RAID nativo</b>                 | Sí (RAID-Z1/Z2/Z3 + mirror)                                  | Sí (RAID 0/1/10; RAID 5/6 experimental)    |
| <b>Checksums de datos</b>          | Sí (256 bits, en árbol)                                      | Sí (CRC32C por defecto)                    |
| <b>Autorreparación</b>             | Sí (con redundancia)                                         | Sí (con RAID configurado)                  |
| <b>Copy-on-Write</b>               | Sí                                                           | Sí                                         |
| <b>Snapshots</b>                   | Sí (datasets y zvols)                                        | Sí (subvolúmenes)                          |
| <b>Subvolúmenes</b>                | Datasets (similar)                                           | Sí (nativo)                                |
| <b>Compresión transparente</b>     | Sí (LZ4, GZIP, ZSTD)                                         | Sí (zlib, lzo, ZSTD)                       |
| <b>Deduplicación</b>               | Sí (inline, costosa en RAM)                                  | Solo offline                               |
| <b>Desfragmentación</b>            | No (requiere migración de datos)                             | Sí (en línea, sin desmontar)               |
| <b>Modo SSD</b>                    | L2ARC, opciones de caché                                     | Opción de montaje <code>ssd</code>         |
| <b>Consumo de RAM</b>              | Alto (ARC, dedup)                                            | Moderado                                   |
| <b>Madurez empresarial</b>         | Muy alta                                                     | Media-alta                                 |

## 4.2. Integridad de datos

Ambos sistemas implementan *checksums* para detectar la corrupción silenciosa, pero con diferencias de alcance:

- **ZFS** almacena el *checksum* de cada bloque en el puntero del bloque padre, formando un árbol de Merkle desde la hoja hasta la raíz del pool. Esto garantiza que cualquier modificación maliciosa o accidental de datos o metadatos sea detectada.
- **Btrfs** también verifica datos y metadatos, y mantiene dos copias de los metadatos por defecto. Sin embargo, la arquitectura del árbol B hace que la verificación sea parcialmente dependiente de la configuración RAID para la recuperación efectiva.

En ambos casos, la autorreparación automática solo es posible cuando existe redundancia (mirror, RAID-Z, RAID 1, etc.).

## 4.3. Snapshots y gestión del espacio

- Ambos sistemas crean instantáneas en tiempo casi nulo y con costo de espacio inicial prácticamente cero gracias al CoW.
- **ZFS** distingue entre *snapshots* (solo lectura) y *clones* (escribibles derivados de un snapshot).
- **Btrfs** permite *snapshots* escribibles directamente, lo que simplifica el flujo de trabajo para rollbacks y entornos de prueba.
- Synology aprovecha los snapshots de Btrfs para ofrecer hasta **256 puntos de recuperación** con mínimo impacto en el rendimiento.

## 4.4. Integración con Linux

Esta es la diferencia práctica más importante para usuarios de Linux:

- **Btrfs** está incluido directamente en el árbol del kernel Linux. No requiere módulos adicionales, y su desarrollo está sincronizado con el ciclo de lanzamiento del kernel.
- **ZFS** debe instalarse como módulo externo (**zfs-dkms** o **zfs-linux**), lo que implica pasos adicionales de compilación en cada actualización de kernel, posibles retrasos y limitaciones en distribuciones con restricciones de licencia (como Debian en su configuración más estricta).

#### 4.5. RAID: ZFS vs Btrfs

- **RAID-Z de ZFS** es considerado maduro y confiable para producción. No tiene el defecto del *write hole* y soporta paridad simple (Z1), doble (Z2) y triple (Z3).
- **RAID 5/6 de Btrfs** tiene problemas conocidos con el *write hole* y no es recomendado para producción crítica. RAID 1 y RAID 10 son estables.
- Para entornos que requieren RAID con paridad en Linux, ZFS (via `zfs on linux`) es la opción más robusta actualmente.

#### 4.6. Escalabilidad y consumo de recursos

- **ZFS** escala mejor en entornos de muy alta capacidad (petabytes o más) y cargas de I/O intensas, pero tiene un consumo de RAM significativamente mayor. El ARC puede consumir la mayor parte de la memoria disponible para maximizar el rendimiento.
- **Btrfs** es más ligero en consumo de memoria y más adecuado para sistemas con recursos limitados (NAS domésticos, laptops, servidores pequeños).

#### 4.7. Casos de uso recomendados

| Caso de uso                           | ZFS                   | Btrfs              |
|---------------------------------------|-----------------------|--------------------|
| NAS empresarial de alta capacidad     | <b>Recomendado</b>    | Aceptable          |
| Servidor de virtualización (Proxmox)  | <b>Recomendado</b>    | Posible            |
| Escritorio Linux con rollbacks del SO | No óptimo             | <b>Recomendado</b> |
| NAS doméstico / SOHO                  | Posible               | <b>Recomendado</b> |
| Distribución Linux estándar           | Módulo externo        | <b>Nativo</b>      |
| Entorno FreeBSD / TrueNAS             | <b>Nativo</b>         | No disponible      |
| RAID con paridad en producción        | <b>RAID-Z estable</b> | RAID 5/6 exp.      |
| Deduplicación intensiva               | Posible (RAM alta)    | Ext. (offline)     |

## 5. Fichas de referencia rápida

Las siguientes fichas condensan la información más relevante de cada sistema de archivos en formato de formulario, incluyendo las respuestas breves solicitadas.

### ZFS - Ficha de referencia rápida

|                                               |                                                                                                                                                                                                                                                         |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Año aproximado</b>                         | 2001 (inicio) / 2005 (lanzamiento en OpenSolaris)                                                                                                                                                                                                       |
| <b>Abierto / Propietario</b>                  | Código abierto bajo licencia CDDL (OpenZFS)                                                                                                                                                                                                             |
| <b>SO donde más se usa</b>                    | FreeBSD, TrueNAS, illumos; con módulo externo en Linux (Ubuntu, Proxmox)                                                                                                                                                                                |
| <b>Uso típico</b>                             | Servidores empresariales, centros de datos, NAS de alta capacidad, entornos de virtualización críticos                                                                                                                                                  |
| <b>Implementación / Características clave</b> | Sistema de archivos + gestor de volúmenes fusionados; storage pools ( <code>zpool</code> ); checksums en árbol de Merkle; CoW transaccional; RAID-Z (sin write hole); autorreparación; ARC; compresión y deduplicación transparentes; snapshots e zvols |
| <b>Tamaño máx. de archivo</b>                 | 16 Exabytes                                                                                                                                                                                                                                             |
| <b>Tamaño máx. de volumen</b>                 | $\approx 256$ Zebibytes ( $2^{78}$ bytes; prácticamente ilimitado)                                                                                                                                                                                      |

**Btrfs - Ficha de referencia rápida**

|                                               |                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Año aproximado</b>                         | 2007 (anuncio) / 2009 (incorporación al kernel Linux 2.6.29)                                                                                                                                                                                                                                           |
| <b>Abierto / Propietario</b>                  | Código abierto bajo licencia GPL v2 (nativo del kernel Linux)                                                                                                                                                                                                                                          |
| <b>SO donde más se usa</b>                    | Linux exclusivamente (Fedora, openSUSE, Synology DSM)                                                                                                                                                                                                                                                  |
| <b>Uso típico</b>                             | Estaciones de trabajo Linux con rollbacks de SO, NAS comerciales, servidores medianos, entornos de desarrollo                                                                                                                                                                                          |
| <b>Implementación / Características clave</b> | Árbol B con CoW como estructura unificada; sub-volúmenes independientes; snapshots escribibles; integridad de datos + metadatos duplicados; RAID nativo (RAID 1/10 estable; RAID 5/6 experimental); compresión transparente (zstd); desfragmentación en línea; asignación dinámica de inodos; modo SSD |
| <b>Tamaño máx. de archivo</b>                 | 16 Exabytes                                                                                                                                                                                                                                                                                            |
| <b>Tamaño máx. de volumen</b>                 | 16 Exabytes                                                                                                                                                                                                                                                                                            |

---

## 6. Conclusiones

---

ZFS y Btrfs representan dos filosofías distintas pero complementarias para abordar las limitaciones de los sistemas de archivos tradicionales.

**ZFS** es la solución más madura, robusta y probada en entornos de almacenamiento de misión crítica. Su arquitectura de integridad de extremo a extremo, su RAID-Z libre del *write hole* y su escalabilidad prácticamente ilimitada lo convierten en la elección preferida para centros de datos, arreglos NAS empresariales y sistemas donde la integridad de los datos es absolutamente prioritaria. Su principal limitación en el mundo Linux es la incompatibilidad de licencias, que obliga a utilizar módulos externos.

**Btrfs** es la respuesta nativa del ecosistema Linux a la necesidad de un sistema de archivos moderno. Su integración directa en el kernel, sus subvolúmenes para rollbacks del sistema operativo, su ligereza en consumo de recursos y su adopción por distribuciones líderes (Fedora, openSUSE) y fabricantes NAS (Synology) lo posicionan como la opción más práctica para escritorios y servidores medianos bajo Linux. Su talón de Aquiles es la inmadurez de RAID 5/6 y la menor madurez general frente a ZFS en cargas empresariales extremas.

En definitiva, la elección entre ambos depende del contexto: **ZFS para máxima integridad y escala empresarial; Btrfs para máxima integración con Linux y flexibilidad en sistemas de tamaño medio.**